

Лекция 10

Шаблоны и исключения

1 Функция-шаблон

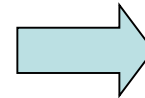
Шаблоны позволяют определить конструкции (функции, классы), которые используют определенные типы, но на момент написания кода точно не известно, что это будут за типы.

Функция-шаблон — это функция без указания точного типа некоторых или всех параметров. При вызове такой функции программист сам определяет типы параметров.

Синтаксис функции-шаблона:

```
template <typename пользовательский_тип>  
возвращаемый_тип имя_функции(список параметров)  
{  
    // тело функции  
}
```

```
1  #include <iostream>
2  using namespace std;
3
4  template <typename type1, typename type2>
5  void myfunc(type1 x, type2 y)
6  {
7      cout << x << "  " << y << endl;
8  }
9
10 int main()
11 {
12     myfunc(10, "hi");
13     myfunc(0.23, 10L);
14     return 0;
15 }
```



```
10  hi
0.23 10
```

```

1  #include <iostream>
2
3  template <typename T>
4  const T& max(const T& a, const T& b)
5  {
6      return (a > b) ? a : b;
7  }
8
9  int main()
10 {
11     int i = max(4, 8);
12     std::cout << i << '\n';
13
14     double d = max(7.56, 21.434);
15     std::cout << d << '\n';
16
17     char ch = max('b', '9');
18     std::cout << ch << '\n';
19
20     return 0;
21 }

```



```

8
21.434
b

```

2 Классы-шаблоны

Шаблон класса позволяет задать тип свойств, используемых в классе.

Синтаксис класса-шаблона:

```
template <typename пользовательский_тип>  
class имя_класса  
{ ... }
```

Создание конкретного экземпляра класса-шаблона:

```
имя_класса <тип> объект;
```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  template <typename T>
6  class MyClass {
7  private:
8      T N;
9  public:
10     MyClass(T n) : N(n) { }
11     T getN() { return N; }
12 };
13
14 int main()
15 {
16     MyClass<int> I(5);
17     cout << I.getN() << endl;
18
19     MyClass<float> F(7.77);
20     cout << F.getN() << endl;
21
22     MyClass<string> S("hello");
23     cout << S.getN() << endl;
24
25     return 0;
26 }

```



```

5
7.77
hello

```

3 Обработка исключений

Общий синтаксис:

```
try { /*блок try*/ }  
catch (тип1 аргумент) { // блок catch }  
catch (тип2 аргумент) { // блок catch }  
...  
catch (типN аргумент) { // блок catch }
```

Инструкция throw должна выполняться либо внутри блока try, либо в функции, вызванной из блока try:

```
throw исключение;
```

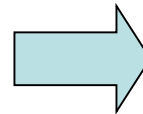
Перехват всех исключений:

```
catch (...)  
{  
    // обработка всех исключений  
}
```

```

1  #include <iostream>
2  using namespace std;
3
4  void F(int test) {
5  try {
6      if (test) throw test;
7      else throw "Это строка";
8  }
9  catch (int i)
10 {
11     cout << "Int: " << i << '\n';
12 }
13 catch(const char *str)
14 {
15     cout << "String: " << str << '\n';
16 }
17 }
18
19 int main()
20 {
21     cout << "Start\n";
22
23     F(1);
24     F(0);
25     F(20);
26
27     cout << "End";
28     return 0;
29 }

```



```

Start
Int: 1
String: Это строка
Int: 20
End

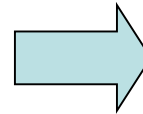
```



```

1  #include <iostream>
2  using namespace std;
3
4  void F(int test) {
5  try {
6      if(test==0) throw test;
7      if(test==1) throw 'a';
8      if(test==2) throw 123.23;
9  }
10 // перехват всех исключений
11 catch (...)
12 {
13     cout << "Исключение!\n";
14 }
15 }
16
17 int main()
18 {
19     cout << "Start\n";
20
21     F(0);
22     F(1);
23     F(2);
24
25     cout << "End";
26     return 0;
27 }

```



```

Start
Исключение!
Исключение!
Исключение!
End

```